ExpenseTracker.jsx

```jsx
import React, { useReducer, useState, useMemo, useEffect } from 'react';
import './ExpenseTracker.css';

// 1. THE REDUCER: Handles complex state transitions predictably.
// Instead of multiple setStates, we dispatch "Actions".
const expenseReducer = (state, action) => {
  switch (action.type) {
    case 'LOAD_DATA':
      return { ...state, transactions: action.payload };
    case 'ADD_TRANSACTION':
      return {
        ...state,
        transactions: [action.payload, ...state.transactions]
      };
    case 'DELETE_TRANSACTION':
      return {
        ...state,
        transactions: state.transactions.filter(t => t.id !== action.payload)
      };
    case 'SET_FILTER':
      return { ...state, filter: action.payload };
    default:
      return state;
  }
};

const initialState = {
  transactions: [],
  filter: 'ALL', // Options: 'ALL', 'INCOME', 'EXPENSE'
};
```

```jsx
export const ExpenseTracker = () => {
  // 2. useReducer: Manages the core application state
  const [state, dispatch] = useReducer(expenseReducer, initialState);

  // 3. useState: Manages local, temporary form inputs ONLY
  const [desc, setDesc] = useState('');
  const [amount, setAmount] = useState('');
  const [type, setType] = useState('EXPENSE');

  // 4. useEffect: Load saved transactions from local storage on first mount
  useEffect(() => {
    const saved = JSON.parse(localStorage.getItem('expense_data'));
    if (saved) {
      dispatch({ type: 'LOAD_DATA', payload: saved });
    }
  }, []);

  // 5. useEffect: Save to local storage whenever transactions change
  useEffect(() => {
    localStorage.setItem('expense_data', JSON.stringify(state.transactions));
  }, [state.transactions]);

  // 6. useMemo: Calculate totals dynamically without looping on every minor render
  const { totalIncome, totalExpense, balance } = useMemo(() => {
    console.log("Calculating Financial Totals...");
    let income = 0;
    let expense = 0;

    state.transactions.forEach(t => {
      if (t.type === 'INCOME') income += t.amount;
      if (t.type === 'EXPENSE') expense += t.amount;
    });
```

```
  return {
    totalIncome: income,
    totalExpense: expense,
    balance: income - expense
  };
}, [state.transactions]); // Only recalculate if the transaction list changes

// 7. useMemo: Filter the visible transactions based on the active filter
const visibleTransactions = useMemo(() => {
  if (state.filter === 'ALL') return state.transactions;
  return state.transactions.filter(t => t.type === state.filter);
}, [state.transactions, state.filter]);

// Handle Form Submission
const handleAdd = (e) => {
  e.preventDefault();
  if (!desc || !amount) return;

  const newTransaction = {
    id: Date.now(), // Generate unique ID
    desc,
    amount: parseFloat(amount),
    type
  };

  // Dispatch the action to the reducer
  dispatch({ type: 'ADD_TRANSACTION', payload: newTransaction });

  // Reset local form state
  setDesc('');
  setAmount('');
```

```
};

return (
  <div className="tracker-wrapper">
    <div className="tracker-card">

      <header className="tracker-header">
        <h2>Financial OS</h2>
        <div className="balance-display">
          <span className="balance-label">Net Balance</span>
          <h1 className={balance >= 0 ? 'text-positive' : 'text-negative'}>
            RWF {balance.toLocaleString()}
          </h1>
        </div>
        <div className="summary-cards">
          <div className="card income-card">
            <span>Income</span>
            <h3>+RWF {totalIncome.toLocaleString()}</h3>
          </div>
          <div className="card expense-card">
            <span>Expense</span>
            <h3>-RWF {totalExpense.toLocaleString()}</h3>
          </div>
        </div>
      </header>

      {/* Form using Local useState */}
      <form onSubmit={handleAdd} className="transaction-form">
        <input
          type="text"
          placeholder="Description (e.g. Server Hosting)"
          value={desc}
```

```
          onChange={(e) => setDesc(e.target.value)}
          className="input-field"
        />
        <div className="form-row">
          <input
            type="number"
            placeholder="Amount"
            value={amount}
            onChange={(e) => setAmount(e.target.value)}
            className="input-field"
          />
          <select value={type} onChange={(e) => setType(e.target.value)} className="input-field select-field">
            <option value="EXPENSE">Expense</option>
            <option value="INCOME">Income</option>
          </select>
          <button type="submit" className="btn-submit">Add</button>
        </div>
      </form>

      {/* Filter Controls triggering Reducer */}
      <div className="filter-controls">
        {['ALL', 'INCOME', 'EXPENSE'].map(f => (
          <button
            key={f}
            onClick={() => dispatch({ type: 'SET_FILTER', payload: f })}
            className={`btn-filter ${state.filter === f ? 'active-filter' : ''}`}
          >
            {f}
          </button>
        ))}
      </div>
```

```
        {/* Transaction List */}
        <ul className="transaction-list">
          {visibleTransactions.map(t => (
            <li key={t.id} className={`transaction-item ${t.type === 'INCOME' ? 'border-income' : 'border-
expense'}`}>
              <div className="t-info">
                <strong>{t.desc}</strong>
                <span className="t-date">{new Date(t.id).toLocaleDateString()}</span>
              </div>
              <div className="t-action">
                <span className={t.type === 'INCOME' ? 'text-positive' : 'text-negative'}>
                  {t.type === 'INCOME' ? '+' : '-'}RWF {t.amount.toLocaleString()}
                </span>
                <button
                  onClick={() => dispatch({ type: 'DELETE_TRANSACTION', payload: t.id })}
                  className="btn-delete-small"
                >
                  ×
                </button>
              </div>
            </li>
          ))}
          {visibleTransactions.length === 0 && (
            <div className="empty-state">No transactions found.</div>
          )}
        </ul>

      </div>
    </div>
  );
};
```

App.js

```
import React from 'react';
import { ExpenseTracker } from './ExpenseTracker';

export default function App() {
  return <ExpenseTracker />;
}
```

ExpenseTracker.css

```
.tracker-wrapper {
  min-height: 100vh;
  background-color: #020617; /* Deep futuristic blue/black */
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 2rem 1rem;
  font-family: system-ui, -apple-system, sans-serif;
  color: #f8fafc;
}

.tracker-card {
  background-color: #0f172a;
  border: 1px solid #1e293b;
  border-radius: 16px;
  width: 100%;
  max-width: 450px;
  box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.7);
  overflow: hidden;
}
```

```css
.tracker-header {
  padding: 1.5rem;
  background-color: #1e293b;
  border-bottom: 1px solid #334155;
}

.tracker-header h2 {
  margin: 0 0 1rem 0;
  font-size: 1rem;
  text-transform: uppercase;
  letter-spacing: 0.1em;
  color: #94a3b8;
}

.balance-display {
  margin-bottom: 1.5rem;
}

.balance-label {
  font-size: 0.875rem;
  color: #94a3b8;
}

.balance-display h1 {
  margin: 0.25rem 0 0 0;
  font-size: 2.5rem;
}

.text-positive { color: #10b981; }
.text-negative { color: #ef4444; }

.summary-cards {
```

```
  display: flex;
  gap: 1rem;
}

.card {
  flex: 1;
  padding: 1rem;
  border-radius: 8px;
  background-color: #0f172a;
}

.card span {
  font-size: 0.75rem;
  text-transform: uppercase;
  color: #94a3b8;
}

.card h3 {
  margin: 0.5rem 0 0 0;
  font-size: 1.25rem;
}
.income-card h3 { color: #10b981; }
.expense-card h3 { color: #ef4444; }

.transaction-form {
  padding: 1.5rem;
  border-bottom: 1px solid #1e293b;
}

.input-field {
  width: 100%;
  background-color: #020617;
```

```
  border: 1px solid #334155;
  color: white;
  padding: 0.75rem;
  border-radius: 6px;
  margin-bottom: 0.75rem;
  font-size: 1rem;
  box-sizing: border-box;
}
.input-field:focus {
  outline: none;
  border-color: #38bdf8;
}

.form-row {
  display: flex;
  gap: 0.5rem;
}

.select-field { width: 40%; }
.btn-submit {
  background-color: #38bdf8;
  color: #020617;
  border: none;
  border-radius: 6px;
  font-weight: bold;
  padding: 0 1rem;
  cursor: pointer;
  margin-bottom: 0.75rem;
}

.filter-controls {
  display: flex;
```

```
  gap: 0.5rem;
  padding: 1rem 1.5rem;
  background-color: #1e293b;
}

.btn-filter {
  flex: 1;
  background-color: transparent;
  color: #94a3b8;
  border: 1px solid #475569;
  padding: 0.5rem;
  border-radius: 4px;
  cursor: pointer;
  font-size: 0.75rem;
}
.active-filter {
  background-color: #475569;
  color: white;
  border-color: #64748b;
}

.transaction-list {
  list-style: none;
  padding: 0 1.5rem 1.5rem 1.5rem;
  margin: 0;
  max-height: 300px;
  overflow-y: auto;
}

.transaction-item {
  display: flex;
  justify-content: space-between;
```

```
  align-items: center;
  background-color: #1e293b;
  padding: 1rem;
  border-radius: 8px;
  margin-top: 0.75rem;
  border-left: 4px solid transparent;
}
.border-income { border-left-color: #10b981; }
.border-expense { border-left-color: #ef4444; }

.t-info {
  display: flex;
  flex-direction: column;
}
.t-date {
  font-size: 0.75rem;
  color: #64748b;
  margin-top: 0.25rem;
}

.t-action {
  display: flex;
  align-items: center;
  gap: 1rem;
  font-weight: bold;
}

.btn-delete-small {
  background-color: #7f1d1d;
  color: white;
  border: none;
  border-radius: 4px;
```

```
  width: 24px;
  height: 24px;
  display: flex;
  justify-content: center;
  align-items: center;
  cursor: pointer;
}
.btn-delete-small:hover { background-color: #991b1b; }

.empty-state {
  text-align: center;
  color: #64748b;
  padding: 2rem 0;
  font-style: italic;
}
```